Final Review for second half

2025年6月4日 18:23

Complex System

A model is a simplified version of Reality, it has some assumptions. These assumptions may not have great impact on creating the design. But useful when report findings using the model, and readers can use that assumption and result.

Parts, rules, behaviour
Emergent: special behaviour, not being designed. Therefore, an orchestra is not as complex as its outcome of a centralised commander

Complex systems. . .
- are made up of a number of components. . .
- that interact with each other. . .
- typically in a non-linear fashion.

Complex systems. . .
- may arise from and evolve through self-organization. . .
- are neither completely regular nor completely random. . .
- permit the development of emergent behaviour at macroscopic scales.

Complex System features

Emergence
- The system has properties that the individual parts do not
- These properties cannot be easily inferred or predicted
- Different properties can emerge from the same parts, depending upon context or arrangement

Self-organisation
- Order increases without external intervention
- Typically as a result of interactions between parts

Decentralisation
- There is not a single controller or 'leader'
- distribution: each part carries a subset of global information
- bounded knowledge: no part has a full view of the whole
- parallelism: parts can act simultaneously

Feedback
- Positive feedback amplifies fluctuations in system state
- Negative feedback dampens fluctuations in system state

negative* feedback acts to make system behaviour more *stable*, while *positive* feedback makes system behaviour more *unstable*

Agent Based Model ABM
Agents: with local state and rules
Interaction: interaction between agents, normally about update rules
Rules/Behaviour: next state would depend on interaction

Dynamic System
Consist of
Set of States
Time
Initial State X0 at Time t0
A rule determine Xn at tn

Logistic Model
xt+1 = rxt(1 − xt)
X = P/A ,where P is population and A is food or resource. --> When P reach to limit, population would collapse, if P/A =0, the equation would looks like exponential with Xt+1 = r(Xt).
Rxt could be positive feedback as it indicates the population would grow large, and (1-xt) is negative as its controlling the growth
0<r<1 -> population die out
1<r<3 -> A stable point near 0.5
r>3 -> no true stable point, looks like a Sine Curve
When r is too large, like r > 4 . It becomes chaos with looping and no structure.

Chaotic
- The dynamical update rule is deterministic
- The system behaviour is aperiodic
- The system behaviour is bounded
- The system displays sensitivity to initial conditions

Key Structure
- fixed points
- limit cycles
- aperiodic orbits

Lotka-Volterra model

Model formulation

Prey (rabbits): $\frac{dR}{dt} = \alpha R - \beta RF$

Predators (red foxes): $\frac{dF}{dt} = \delta RF - \gamma F$

Parameters:

Parameter	*Meaning*
α	growth rate of the rabbit population
β	rate at which foxes predate upon (eat) rabbits
δ	growth rate of the fox population
γ	decay rate of the fox population due to death and migration

Equilibrium point at dR/dt = 0 and dF/dt = 0 together --> either
Both extinction,
Form a circle.

Lotka-Volterra model with competition

$$\frac{dR}{dt} = \alpha R - \beta RF - aR^2$$
$$\frac{dF}{dt} = \delta RF - \gamma F - bF^2$$

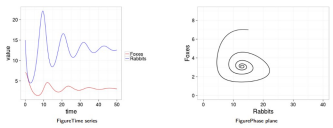


Figure1Time series

Figure2Phase plane

SIR model
Susceptible, infectious, recovered model, a very basic disease model.
Detail:
A S is near an I, it may become I probability = p
An I can become R probability = q
Each Day:
Get a random value x, x > p then infect that S, repeat for all S
In more realistic modelling, each S may meet K I, therefore true prob would be p^k

Get a random value y, y > q then recover that I, repeat for all I
Until I = 0

With these probs, there is no need to calculate each agent in the model. Can use math equation instead.
Each day:
Infect 1- (1-p) ^k S people,
Recover q*I people
Until I = 0
So it can use on very large population

Stochastic model
Generates distribution of models
incorporate the effects of chance
Results are different each time we run. -> random seed is different each time.
Estimate the most average future, but averaging the random results.

Deterministic model
Generates unique outcomes
Based on the parameter and initial conditions
Infection rate R0
Chance of influence in each generation.
R0>1 -> outbreak
R0<1 -> die out
SIR model in deterministic

St+1 = St − βSt It It+1 = It + βSt It − γIt Rt+1 = Rt + γIt		dS/dt = −βSI dI/dt = βSI − γI dR/dt = γI

β and γ are the rates of effective contact (per capita) and recovery.
Finally use R/P -> total infected.
Vaccine -> When V > 1 - 1/R0, Vaccine is effective
Extension of SIR model with demography

dS/dt = mN − βSI − mS
dI/dt = βSI − γI − mI
dR/dt = γI − mR
Where m is the new born and die rate. This makes the population keep refreshing while keep the same number of people -> S+I+R = N

With Vaccination
dS/dt = m(1 − v)N − βSI − mS
dI/dt = βSI − γI − mI
dR/dt = mvN + γI − mR

Cellular Automata
Discrete dynamic system
Complex behaviour, simple and local rules.

Automaton
a machine (cell in our case) update its internal states based on external input and self previous state.
Cellular Automaton
An array of automatons where input to one cell are the states of nearby cells
Feature
Time and Space is discrete
System state is discrete
Rules are defined in terms of local interactions between components

1D:

To define a transition function Δ, a unique next state in Σ must be associated with a possible neighbourhood state.

Since there are $K = |\Sigma|$ choices of states to assign as the next state for each of the $|\Sigma^N|$ possible neighbour states, there are K^{K^N} possible transitions functions Δ that can be defined.

D_N^K is the set of all possible transitions functions Δ which can be defined using N neighbours and K states.

Conway's Game of Life
2D grid
8 neighbours
Rules:
Any live cell
N<2 die
N=2,3 live
N>3 die
Any dead cell
N=3 live

With this, can perform amazing anims

Lotka-Volterra Predator Prey model

Fox and rabbit.

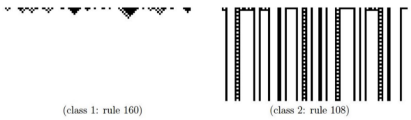
Fox eat rabbit. More fox, less rabbit --> less fox --> more rabbit and repeat

$|\Sigma^{N}|$ possible neighbour states, there are $K^{|\Sigma^{N}|}$ possible transitions functions Δ that can be defined.

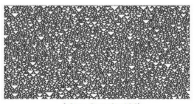
D_N^K is the set of all possible transitions functions Δ which can be defined using N neighbours and K states.

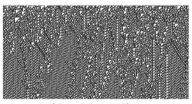
Consider a 2-dimensional cellular automata using 8 states per cell; a regular lattice with a five cell local neighbourhood (north, south, east, west).
Here, $K = 8$ and $N = 5$, so there are:
$|\Delta| = K^N = 8^5 = 32768$ possible neighbourhood states.
For all of these neighbouhood states, there is a choice of 8 different states for the next cell state under Δ, so there are:
$D_N^K = K^{K^N} = 8^{8^5} \approx 10^{30000}$ possible transitions functions.

In a Cellular Automaton, its same as previous math model, we observe for:
- Transient
- Fixed point
- Limit cycle


(class 1: rule 160)

(class 2: rule 108)


(class 3: rule 126)


(class 4: rule 110)

2D
Different in neighbourhood
- Von Neumann neighbourhood
  - Only four direction, up,down,left,right
- Moore neighbourhood
  - Eight direction, cycle

Key features to assumption in 2D CA Have done this using Netlogo
- Space, grid and each agent takes one grid space
- Time, discrete time step
- Information, neighbour grids
- State Updating, specific rules for each agent

Parameter Sweep
- Modify each parameter to see their effect on the system
- Run several time to calculate average as system is stochastic
- For trajectory, observe some global statics.

CA extension

Asynchronous CA
The basic CA updates *all* cells synchronously (at the same time) at *every* time step. We could also update cells at different times.

Probabilistic CA
The basic CA update rules are deterministic (Cx.05 examples). We could also use update rules that are probabilistic/stochastic (Cx.06 examples).

Non-homogeneous CA
The basic CA applies the same update rule to every cell. We could also use context-sensitive rules.

Network-structured CA
The basic CA defines neighbourhood in terms of grid adjacency. We could also use a more complex *network* topology of neighbours (we'll look at this later)

Why use CA
- Advantage
  - Simple and easy to implement
  - Represent interaction and behaviours that are difficult to model
  - Reflect intrinsic individual of system
- Disadvantage
  - Relatively constrained with topology, interactions and individual behaviour
  - Global behaviour maybe hard to interpret

Agent Based Model ABM
- Features
  - Agent
  - Environment
  - Interaction
- Essential Agent Characteristic
  - Self-contained
    - An agent is a modular component of the system
    - Has boundary and recognized by other agents
    - E.g. Birds in flocking model大雁飞人字
  - Autonomous
    - Function independently in its environment and in its interaction with other agents
    - E.g behaviour of each boid is only depends on itself and its neighbour
  - Dynamic State
    - Attributes, or variables deciding its future, an agent's current state is what determine its future.
  - Social
    - Dynamic interaction with other agents that influence their behaviour
    - E.g ant communication
- Other Characteristic
  - Adaptive
    - May have the ability to learn from past
  - Goal-directed
    - May have goals attempting to achieve via behaviour
  - Heterogeneous
    - May contain different agents (pred and prey)
- Environment
  - Agents monitor and reacts to the environment. A physical or virtual space contain agents
  - The grid of CA is simple environment

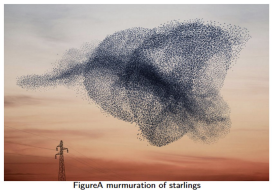
Fox and rabbit.

Fox eat rabbit. More fox, less rabbit --> less fox --> more rabbit and repeat

Difference between ABM and CA

ABM has more freedom in agent behaviour than in CA
ABM can have individual agent behaviour, in CA, the rule is set to be the same among the grid

flocking behaviour


FigureA murmuration of starlings

Large group of bird or fish
Each agent (boid) behaves
- Front
- Turning left or right
- Acceleration and Deceleration

Agent Feature
- Reflexive
- Internal state of speed and heading(velocity)
- No goal, no past experience,

Minimal Agent

Neighbour:
- Other agents in vision

Interaction
- Separation: Try to maintain a discuss
- Cohesion: Try to stay together
- Alignment: Try to stay in same direction

Foraging in ant colony

Simple ant behaviour
- Pick or drop food and construct material

Interaction is pheromone 信息素 from other ants
In a bridge problem, ants choose the shorter path to go.
As more pheromone is deployed on the road, attracting more ants to go through the path and drop pheromone.
This is a kind of positive feedback

May contain different agents (pred and prey)

Environment

Agents monitor and reacts to the environment. A physical or virtual space contain agents
The grid of CA is simple environment
Environments maybe static, or change by agent behaviour, or independently dynamic
Real environment generally dynamic and stochastic
Richer environment may include

Continuous, in two to three dimension
Multiple layers
Complex feedback

Interactions

Local interaction within the neighbourhood of agent
The neighbourhood maybe dynamic, changed when agent behaviour, like when agent moves
Not always in spatial terms, sometimes network with particular topology

Agent decision making

Agents are often designed to make rational decisions: they will act to maximise the expected value of their behaviour given the information they have received thus far.
Note that:

- rational/= omniscient: an agent's percepts may not supply all required information
- rational/= clairvoyant: an agent's actions may not produce the expected outcome
- therefore, rational/= successful (necessarily)
- rational → exploration, learning, adaptation, . . .

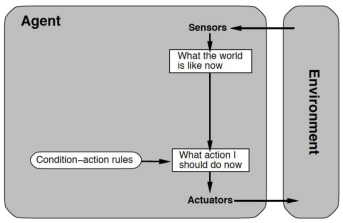
Agent decision making may be:

- probabilistic: representing decisions using distributions
- rule-based: modelling the decision making process
- adaptive: agents may learn via reinforcement or evolution

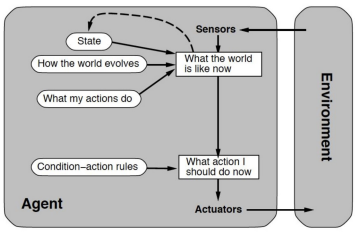
The choice will often depend on the purpose of the model (what question is it being used to answer?)
Agent decision making can range from simple to complex. . .

Use a KISS principle, keep it simple and stupid

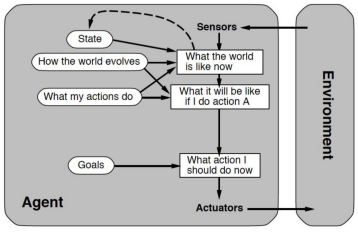
Reflexive



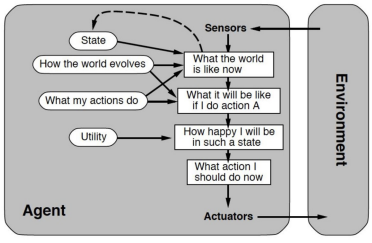
Reflexive with internal state



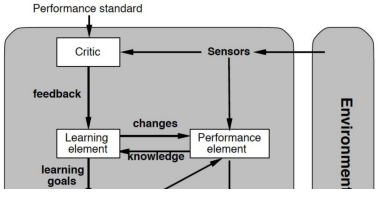
Goal Driven



Utilitarian



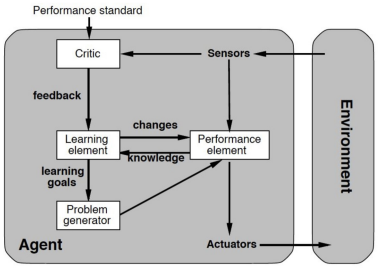
Learning



through the path and drop pheromone.
This is a kind of positive feedback

Algorithm finding the shortest path

1. Distribute N ants at random among the nodes (destinations)
2. Each ant randomly chooses a new node to travel to, based on:
   - pheromone concentration on the edge to that node (weighted by α)
   - distance to that node (weighted by β)
   - it must not have visited that node previously
3. Repeat step 2 until each ant has visited all nodes
4. Calculate the total route length for each ant
5. The ant that achieved the shortest route deposits pheromone along the edges in their route (with concentration inversely proportional to route length)
6. Pheromones evaporate at a rate ρ



Petri Nets

How it Begin

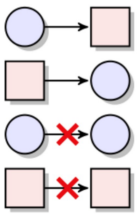
- Hard to calculate recursive function
- Recursive algorithms needs to expand if current resource is not enough to perform calculation
- Expanding system is cheaper than have a new one
- Categorized as
  - No central component
  - No central synchronising block

A Petri Net contains

- Places

  - Place P is a passive system component
  - Has discrete states and can store things
- Transition

  - Transition t is active system component
  - Consume, produce or transfer things
- Arcs



  - Not a system component but a logical relation between components
  - An arc relates a place to a transition or a transition to a place

$$F \subseteq (P \times T) \cup (T \times P).$$

And net =

$$N = (P, T, F)$$

is a net structure. Places and transitions are the ***elements*** of N, and F is the ***flow relation*** of N.

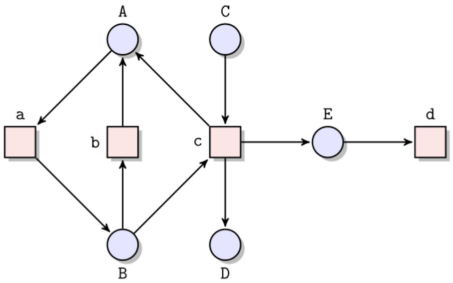


Figure: A net structure (P, T, F) with $P = \{\mathrm{A}, \mathrm{B}, \mathrm{C}, \mathrm{D}, \mathrm{E}\}$, $T = \{\mathrm{a}, \mathrm{b}, \mathrm{c}, \mathrm{d}\}$, and $F = \{(\mathrm{A}, \mathrm{a}), (\mathrm{a}, \mathrm{B}), (\mathrm{B}, \mathrm{b}), (\mathrm{b}, \mathrm{A}), (\mathrm{B}, \mathrm{c}), (\mathrm{c}, \mathrm{A}), (\mathrm{C}, \mathrm{c}), (\mathrm{c}, \mathrm{D}), (\mathrm{c}, \mathrm{E}), (\mathrm{E}, \mathrm{d})\}$.

Marking

- Token
  - A real-world thing. Normally just use black dot in Ordinary net system

Ordinary net System

- S pair of (N,M0) where N is the net with finite, disjoint, sets of place and transitions M0 is an initial marking

Pre-set and post-set of an element

Given a net structure (P, T, F), the pre-set $^\bullet x$ and the post-set $x^\bullet$ of an element $x \in P \cup T$ are defined as follows:

$$^\bullet x =_{\text{def}} \{y \in P \cup T \mid (y, x) \in F\} \quad \text{and}$$
$$x^\bullet =_{\text{def}} \{y \in P \cup T \mid (x, y) \in F\}.$$

Pre-set and post-set of an element

Given a net structure (P, T, F), the pre-set $\bullet x$ and the post-set $x\bullet$ of an element $x \in P \cup T$ are defined as follows:

$$\bullet x =_{\text{def}} \{y \in P \cup T \mid (y, x) \in F\} \quad \text{and}$$
$$x\bullet =_{\text{def}} \{y \in P \cup T \mid (x, y) \in F\}.$$

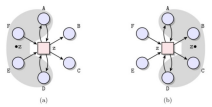


Figure: (a) Pre-set $\bullet z = \{A, D, E, F\}$ and (b) post-set $z\bullet = \{A, B, C, D\}$ of transition z.

The transition enablement rule

Given a net system (N, M), where $N = (P, T, F)$, marking M enables transition $t \in T$ if every place in the pre-set of t contains at least one token (i.e., $\forall\, p \in \bullet t\colon M(p) \geq 1$).

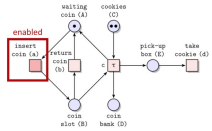


Figure: A net system and its only enabled transition a.

Note in this figure, there is no marking at B, so the preset of b and c is not full, so they are not enabled

The step rule

An enabled in marking M transition t can *occur*. An ***occurrence*** of t results in a step $M \xrightarrow{t} M'$, where marking M' is the next state of the system obtained from M by first destroying one token in every place in the pre-set of t and then creating one fresh token in every place in the post-set of t.

The number of tokens at place $p \in P$ in marking M' is defined as follows:

$$M'(p) =_{\text{def}} \begin{cases} M(p) - 1 & p \in \bullet t \wedge p \notin t\bullet \\ M(p) + 1 & p \in t\bullet \wedge p \notin \bullet t \\ M(p) & \text{otherwise.} \end{cases}$$

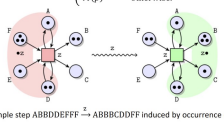


Figure: An example step ABBDDEFFF $\xrightarrow{z}$ ABBBCDDFF induced by occurrence of transition z.

Hot and cold transitions

So far, we have assumed that an enabled transition will either eventually become disabled or occur. This is indeed true for transitions "return coin (b)" and "c", which are internal to the system. For example, a coin in the coin slot will eventually be transferred to the coin bank or returned. We refer to such transitions as hot transitions.

However, in the real-world, transitions "insert coin (a)" and "take cookie (d)" are triggered by a customer (an external system), and it is not guaranteed whether they will ever occur. We refer to such transitions as cold transitions and denote them with ε.

Cold transitions are rare and are often found at the interface to the environment.[2]

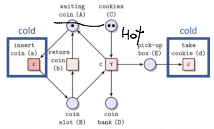


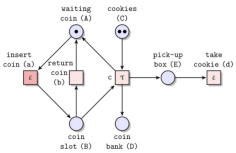
Figure: Cold transitions of the cookie vending machine.

Reachable markings

A possible state of system

Final markings

When no hot transitions are enabled -> the system can just stay there forever



Marking	Reachable	Final
BCC	✓	✗
ACC	✓	✓
ADD	✓	✓
ACDDE	✗	✓
ABCDE	✗	✗

Sequential runs (occurrence sequences)

A sequential run of a net system $S = (N, M_0)$ is a possibly infinite sequence of steps that starts at the initial marking M_0 of S.

$$M_0 \xrightarrow{t_1} M_1 \xrightarrow{t_2} M_2 \xrightarrow{t_3} \cdots$$

We are often interested in ***complete sequential runs***:

- A finite sequential run is complete if it leads to a *final marking* of the net system
- An infinite sequential run is complete if no additional step can be inserted into it

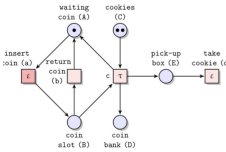


Figure: A net system.

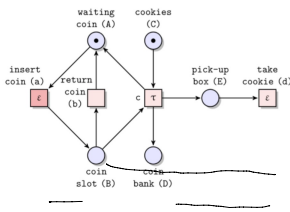
Sequential run	Finite	Complete
$\text{ACC} \xrightarrow{a} \text{BCC}$	✓	✗
$\text{ACC} \xrightarrow{a} \text{BCC} \xrightarrow{b} \text{ACC}$	✓	✓
$\text{ACC} \xrightarrow{a} \text{BCC} \xrightarrow{c} \text{ACDE}$	✓	✓
$\text{ACC} \xrightarrow{a} \text{BCC} \xrightarrow{c} \text{ACDE} \xrightarrow{a} \text{BCDE} \xrightarrow{c} \text{ADDEE} \xrightarrow{d} \text{ADDE} \xrightarrow{a} \text{ADD} \xrightarrow{b} \text{BDD} \xrightarrow{d} \text{ADD} \xrightarrow{b} \text{BDD} \rightarrow \cdots$	✗	✓
$\text{ACC} \xrightarrow{a} \text{BCC} \xrightarrow{b} \text{ACC} \xrightarrow{a} \cdots$	✗	✗

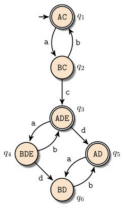
Marking graphs

Nodes are states

Edge are steps

Initial is highlight with arrow pointing, final is highlight with double line boarder





Interleaving semantics of net systems

The collection of all *complete sequential runs* of a net system defines the interleaving semantics of the system.

- A net system can describe an infinite collection of *complete sequential runs*
- Each *sequential run* of a net system is a directed walk in its *marking graph* that starts at the node that corresponds to the initial marking of the net system
- The *marking graph* describes all and only complete sequential runs of the net system

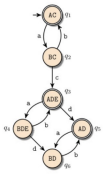


Figure: The marking graph of S.

1. The empty sequence
2. $AC \xrightarrow{a} BC \xrightarrow{b} AC$
3. $AC \xrightarrow{a} BC \xrightarrow{b} AC \xrightarrow{a} BC \xrightarrow{b} AC$
   …
n. $AC \xrightarrow{a} BC \xrightarrow{c} ADE$

n+1. $AC \xrightarrow{a} BC \xrightarrow{b} AC \xrightarrow{a} BC \xrightarrow{c} ADE$
   …
m. $AC \xrightarrow{a} BC \xrightarrow{c} ADE \xrightarrow{d} AD$

m+1. $AC \xrightarrow{a} BC \xrightarrow{b} AC \xrightarrow{a} BC \xrightarrow{c} ADE \xrightarrow{d} AD$
   …
k. $AC \xrightarrow{a} BC \xrightarrow{b} AC \xrightarrow{a} BC \xrightarrow{c} ADE \xrightarrow{d} AD \xrightarrow{a} BD \xrightarrow{b} AD \xrightarrow{a} \cdots$

State Explosion happens when trying to list all states with multiple markings

The state explosion problem

In general, a net system can define a very large, or even infinite, marking graph. This phenomenon is known as the state explosion problem.[1]

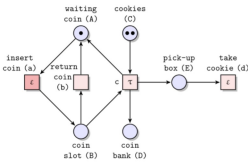


Figure: A two cookies vending machine S'.

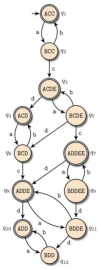
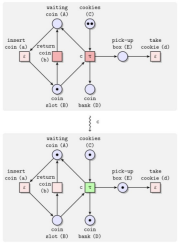


Figure: The marking graph of S'.

Actions

A net system can be interpreted as a collection of ***distributed runs***. A distributed run describes the behaviour of the system more accurately but requires more effort to understand. The building block of a *distributed run* is an action, which describes a *step*.



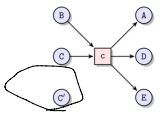


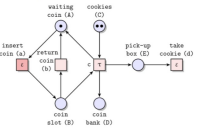
Figure: An action that describes the step on the left

- The transition refers to the occurred transition
- The places without incoming arcs represent the *marking* before the transition occurrence
- The places without outgoing arcs represent the *marking* after the transition occurrence

C' here means reusing the C place, indicating the second cookie being bought

Distributed runs (processes)

A distributed run of a net system is a possibly infinite acyclic net structure, also called a ***causal net***, that starts at the initial marking and is defined by an *uninterrupted sequence of actions* executed by the system.



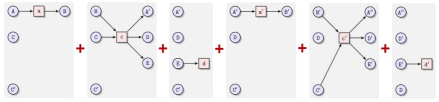


Figure: An uninterrupted sequence of actions of the two cookies vending machine.

Therefore, the second cookie is triggered once the first cookie moves out C

Concurrency semantics of net systems

Each transition of a distributed run represents an *action*. The collection of all ***complete distributed runs*** of a net system defines the concurrency semantics of the system.

- A finite distributed run is complete if its places without outgoing arcs represent a *final marking* of the net system
- An infinite distributed run is complete if no additional action can be inserted into it

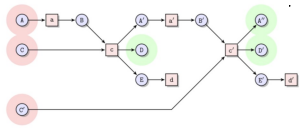
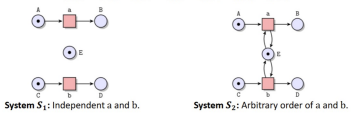


Figure: A complete distributed run from marking ACC to marking ADD.

Sequential vs distributed runs

Both systems shown below have the same complete sequential runs[2], namely:

$$ACE \xrightarrow{a} BCE \xrightarrow{b} BDE \quad \text{and} \quad ACE \xrightarrow{b} ADE \xrightarrow{a} BDE.$$



System S_1: Independent a and b.

System S_2: Arbitrary order of a and b.

However, complete distributed runs of these two net systems are distinct.

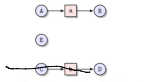


Figure: The only complete distributed run of S_1.

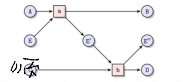


Figure: One of the two complete distributed runs of S_2.

13 /

Causality and concurrency relations

Distributed runs encode ***causality*** *and* ***concurrency*** relations between actions.

A chain of arcs from element x to element y in a causal net specifies that x causally precedes y, denoted by $x \mapsto y$.

If two elements x and y of a distributed run are not causal (i.e., neither $x \mapsto y$ nor $x \mapsto y$ holds), then they are concurrent, denoted by $x \,||\, y$.

The causality, inverse causality, and concurrency relations partition the Cartesian product of elements of a distributed run.

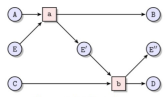


Figure: A distributed run.

	a	b	A	B	C	D	E	E'	E''
a	#	↦	↤	↦	#	↦	↦	↦	↦
b	↤	#	↤	#	↤	↦	↤	↤	↤
A	↦	↦	#	↦	#	↦	#	↦	↦
B	↤	#	↤	#	#	#	↤	#	#
C	#	↦	#	#	#	↦	#	#	↦
D	↤	↤	↤	#	↤	#	↤	↤	#
E	↦	↦	#	↦	#	↦	#	↦	↦
E'	↤	↦	↤	#	#	↦	↤	#	↦
E''	↤	↤	↤	#	↤	#	↤	↤	#

Boundedness

A net is bounded if all reachable markings are finite.(Does not falls into infinite meaningless loop like keep insert and return coin in the cookie system)

The example cookie system is bounded

K-Boundedness

All place P in reachable marking M only contain tokens <=k

The cookie example is therefore 2-bounded

A K-bound is also M-bound where M is integer and M > k

1-bound is called safe

Liveness

To determine a transition is Live, for each reachable markings, there is a way to enable the transition.

Feature:

The transition can be activate again and again

System is Live if all transition is live

If a system is not live, it is possible that some transition can only enable once

For the example cookie system, transition c is enable only once as there are only two cookies and can only wait for a second coin to toss. Transition D is twice as only two cookies can be collected.

Deadlock-freedom

For each reachable marking, enables at least one transition

A deadlock is a reachable marking of the system in which no transition is enabled

Deadlock can be use to identify the termination of system.

Workflow net

Is a net with empty preset i and empty postset o. Source place and sink place

All element would be on the vertex from i to o --> this is more like a function that takes parameter at i and return something like o

To execute, place a token at i, call it a case.

A case indicate all tokens placed at that time, cases can run concurrently

All tokens of a reachable marking describe a reachable state of the workflow system

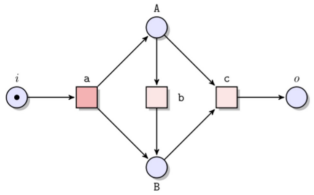
Soundness of Workflow

Every case eventually terminates -> Only a token at sink and no token in any other place for that case

No dead transition -> for every transition t, there is a case that execute t

Option to Complete

For every reachable marking, there is a way to put a marking at sink place o



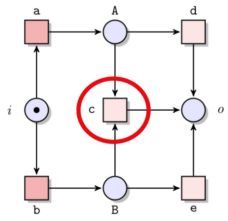
This example may lead to BB state, this time,as A is empty, can not execute c as not all preset has token. Hence not to o, therefore, the reachable BB is not complete ?

Proper Completion

With only one token at sink place o, and no token else where

No dead transition

All transitions, there is at least one reachable marking to enable it



In this example, the token can only go through a or b and stay at A or B. To make c enable, both preset A,B should have token, thus c is never enabled.

Soundness

- Only when workflow satisfy all three above
- Option to Complete
- Proper Completion
- No dead transition

Short circuiting of workflow net

- Connecting the sink place o with initial place i with t*. Where t* not in T
- Compared to original net, same Place P, T U t*, F U {(t*, i),(o, t*)}

This is a shortcut for soundness.

A system S is sound only when the short circuit is live and bounded